

2026

**MANUAL DE TRANSFERENCIA Y
SOSTENIBILIDAD
PAUDIT**

1) Para qué sirve este manual y a quién va dirigido

Este documento existe para que cualquier entidad que quiera implantar PAUDIT (Plataforma de Accesibilidad Universal y Dinamización Turística del Patrimonio), un ayuntamiento, una mancomunidad, una oficina de turismo, un centro educativo o una asociación local... pueda hacerlo sin empezar de cero y sin depender de un equipo técnico grande. La idea es que quien lo lea entienda el “cómo” completo: qué se está replicando, cómo se pone en marcha, cómo se publican rutas y cómo se mantiene el sistema con el paso del tiempo.

La transferencia se apoya en una premisa muy práctica: el código se trata como un producto de software (con control de versiones y cambios ordenados), mientras que las rutas se tratan como un producto editorial (con guiones, textos, audios, vídeos, transcripciones y revisión). Por eso, lo que se comparte es un repositorio vivo que se puede clonar, importar en Replit, desplegar y mantener.

El repositorio de referencia es este:
<https://github.com/resolutpro/PAUDIT.git>

A partir de ahí, este manual guía la implantación con un enfoque pensado para la realidad de la administración local: cambios de equipo, proveedores distintos, necesidades de publicar rápido, y la importancia de que el sistema sea fácil de operar cuando ya está en producción.

2) Qué es PAUDIT v1.0 y qué experiencia ofrece

PAUDIT es una plataforma web para publicar y consumir rutas patrimoniales desde el móvil, con una experiencia muy centrada en el uso real: personas caminando, consultando el mapa, entrando a paradas numeradas y reproduciendo contenidos sin fricción. La web está concebida para que el visitante “haga la ruta” de forma natural: primero elige una ruta, después ve sus paradas, y por último entra a cada parada para acceder a los materiales.

Lo importante de PAUDIT no es solo “mostrar información”, sino hacerlo con un estándar de accesibilidad que se note en la práctica. Por eso cada parada se entiende como una unidad completa de contenido, preparada para distintas formas de consumo. En una parada, el usuario puede leer el texto en pantalla, escuchar una audioguía y, si lo necesita, consultar la transcripción completa. Si existe vídeo en Lengua de Signos, también se ofrece y se acompaña de un texto equivalente (transcripción o descripción del contenido signado) para asegurar que la información no queda “secuestrada” en un único formato.

Esta lógica tiene una ventaja clave para la transferencia: una vez que el modelo editorial está claro, crear nuevas rutas no exige “programar”. El sistema está diseñado para que el trabajo habitual sea: preparar contenidos (texto, audio, vídeo, transcripciones, imágenes), cargarlos donde corresponde y publicarlos siguiendo una estructura estable. En otras palabras: la plataforma se mantiene sólida y el contenido es lo que crece.

Además, PAUDIT separa muy bien lo que cambia de lo que no cambia. La interfaz y la lógica de la web se mantienen estables; lo que se actualiza de forma continua son las rutas y paradas. Esto permite que diferentes municipios o enclaves repliquen el sistema con su identidad patrimonial sin entrar en desarrollos costosos: el “molde” es el mismo, lo que cambia es el relato y el material audiovisual.

3) Por qué se usa Replit como plataforma de desarrollo

Replit se elige como entorno de implantación porque resuelve, en un mismo lugar, lo que en muchos proyectos se convierte en una cadena de dolores: dónde se desarrolla, cómo se despliega, cómo se sirve la web al público, cómo se gestionan archivos multimedia, cómo se coordinan personas que no son todas técnicas y cómo se mantiene el sistema sin montar infraestructura propia compleja.

Este enfoque significa que la entidad que replica PAUDIT no necesita diseñar una arquitectura desde cero ni depender de múltiples proveedores desde el minuto uno. Puede importar el repositorio, ejecutar el proyecto, desplegarlo y operar los contenidos con un flujo estable. En un contexto municipal, donde puede haber rotación de personal, recursos limitados o equipos mixtos (técnicos, personal de turismo, validadores de accesibilidad, asociaciones locales), esa reducción de fricción es decisiva: el proyecto no se sostiene por heroísmo técnico, sino por un procedimiento sencillo y repetible.

Además, Replit encaja especialmente bien con PAUDIT por dos necesidades muy concretas. La primera es el despliegue: una web de rutas patrimoniales puede tener picos de visitas (eventos, campañas, fines de semana) y momentos de poco tráfico. Trabajar con un modelo de despliegue que se adapta ayuda a sostener el proyecto sin sobredimensionar infraestructura. La segunda es la gestión de multimedia: PAUDIT no es solo texto; necesita servir imágenes, audios y vídeos de forma fiable para que el usuario no encuentre enlaces rotos ni reproducciones fallidas. Centralizar esto en un entorno donde el despliegue y el almacenamiento están bien integrados reduce errores operativos y hace que la réplica sea realista.

En la práctica, Replit se convierte en la “mesa de trabajo” donde la entidad puede tener el proyecto corriendo, publicar cambios y mantener el servicio. Y eso es exactamente lo que buscamos en una transferencia: no solo entregar código, sino entregar un sistema que otra organización pueda operar con garantías.

4) Cómo está construido PAUDIT por dentro

PAUDIT está planteado como una única aplicación web que, desde el punto de vista de quien la replica, se entiende mejor si piensas en ella como dos piezas que trabajan juntas: la parte visible (la web que usa el visitante) y la parte “de servicio” (un servidor que entrega datos y archivos multimedia de forma fiable).

La web que ve el usuario (el frontend) está hecha para que la experiencia sea cómoda en móvil: listado de rutas, acceso a cada ruta por su identificador (slug), navegación por paradas, mapa con coordenadas cuando existen, y consumo de audio y vídeo sin complicaciones. Esta parte vive en la carpeta `client/`, y en producción se genera como un conjunto de ficheros estáticos (HTML, JS, CSS) que se sirven al navegador.

La segunda pieza es el servidor (el backend) que vive en `server/`. Su trabajo no es “gestionar usuarios” ni “guardar bases de datos” en esta versión, sino algo mucho más orientado a la replicabilidad: actuar como puerta de entrada única para dos cosas críticas:

- Entregar las rutas y sus paradas mediante una API muy simple.
- Servir los archivos multimedia (imágenes, audios, vídeos) desde Replit App Storage, pero con un comportamiento que en la práctica marca la diferencia: que audio y vídeo se reproduzcan bien en móviles y navegadores modernos.

En concreto, el servidor expone endpoints claros. Cuando el frontend necesita el listado de rutas, llama a `GET /api/routes`. Cuando necesita los datos completos de una ruta concreta, llama a `GET /api/routes/:slug`. Esos datos no salen de una base de datos: el servidor los lee de ficheros JSON que están dentro del propio repositorio (en `client/public/content/`). Esto simplifica muchísimo la transferencia, porque para crear o modificar rutas no hace falta “instalar un CMS”; basta con mantener el contenido en ficheros bien formados y versionados.

La parte más importante del servidor, en términos de experiencia real, es el endpoint de multimedia: todo lo que en el contenido aparece como `/api/media/...` acaba llegando aquí. ¿Por qué se hace así y no con enlaces directos? Porque los navegadores, cuando reproducen audio o vídeo, suelen pedir el archivo por fragmentos (peticiones parciales) para poder saltar, pausar o cargar rápido. Si sirves mal esos fragmentos, el usuario lo nota: se corta el audio, el vídeo tarda, o la reproducción falla de forma intermitente. En PAUDIT el servidor descarga el archivo desde App Storage a una carpeta temporal segura y lo sirve con `sendFile`, dejando que el propio servidor gestione correctamente esas peticiones parciales. Resultado: reproducción mucho más estable.

Esta arquitectura, aunque suene técnica, está pensada para una cosa muy concreta: que otra entidad pueda replicar PAUDIT con garantías sin convertirse en administradora de sistemas. El sistema queda reducido a una idea simple: el repositorio contiene el producto y el contenido, y Replit se encarga de ejecutar y publicar, mientras App Storage se encarga de guardar el multimedia.

5) Cómo se pone en marcha desde el repositorio de GitHub

La forma de arrancar PAUDIT está diseñada para ser directa. Se podría o directamente usar el repositorio de github en un nuevo Replit o, si se trabaja en local, descargar el código del repositorio:

```
git clone https://github.com/resolutpro/PAUDIT.git
```

Si es en local una vez descargado, lo normal es instalar dependencias y ejecutar en modo desarrollo. Esto te permite comprobar que todo funciona antes de desplegar:

```
npm install
npm run dev
```

Este comando levanta la aplicación completa en un único puerto (habitualmente el 5000). Es importante entender que aquí no estás levantando “solo el frontend”: el comando inicia el servidor Express y, en desarrollo, integra la parte de Vite para servir la web, de modo que API y cliente quedan coordinados y no necesitas configuraciones extra.

Cuando trabajas en Replit, esta comodidad está ya preparada. El proyecto incluye un fichero `.replit` con el comportamiento por defecto: al pulsar “Run” ejecuta `npm run dev`, y el puerto 5000 se expone como el puerto web del Repl. Es decir: la instancia que replica PAUDIT no necesita “adivinar” cómo arrancarlo; entra, ejecuta y ve la web.

El paso de “tenerlo funcionando” a “tenerlo publicado” se realiza mediante Deployments. En el propio `.replit` queda definido un despliegue tipo autoscale con tres piezas: un comando de build, un directorio público de salida y un comando de ejecución en producción. En la práctica, esto significa que publicar es tan sencillo como desplegar la versión construida del proyecto: Replit compila la app, genera el directorio de salida (`dist/public`) y ejecuta el servidor ya compilado (`dist/index.cjs`). Esa separación entre desarrollo y producción es una de las razones por las que el sistema se mantiene estable: en producción no dependes del entorno “de desarrollo”, sino de un build reproducible.

A partir de ahí, si la entidad necesita un dominio propio (por ejemplo, un dominio municipal), se conecta el dominio al deployment y se configuran los registros DNS en el proveedor del dominio. Operativamente es un paso administrativo: el resultado es que PAUDIT se sirve bajo el dominio oficial con HTTPS, y el público final no necesita saber nada de Replit.

6) Cómo se publican rutas y paradas

La clave de que PAUDIT sea replicable no está solo en “arrancarlo”, sino en poder publicar rutas sin tocar el código. En esta versión, el contenido vive dentro del repositorio en una estructura muy concreta: un fichero índice con el listado de rutas y un fichero por cada ruta con todas sus paradas.

El índice general está en:

- `client/public/content/routes.json`

Ahí se define qué rutas aparecen en la portada/listado. Cada ruta tiene un slug (que es el identificador estable), un título, el municipio, una descripción corta, una imagen de portada, y algunos metadatos útiles como duración o distancia. Ese slug es importante porque no es decorativo: es el nombre por el que el sistema encuentra el detalle.

El detalle de cada ruta vive en:

- `client/public/content/routes/<slug>.json`

Por ejemplo, si en el índice aparece una ruta con slug: "jerez-monumental-arte-sacro", entonces debe existir un fichero llamado `client/public/content/routes/jerez-monumental-arte-sacro.json`. El servidor, cuando recibe una petición `GET /api/routes/:slug`, busca exactamente ese fichero. Además, aplica una limpieza del slug para evitar caracteres raros, lo cual es una medida de robustez: por transferencia conviene mantener slugs simples (letras, números y guiones).

Dentro del fichero de una ruta, el elemento central es el array `stops`. Cada parada está construida para que el contenido sea accesible “de verdad”, no solo bonito. Por eso aparecen campos que no se deben tratar como opcionales en una réplica seria: `imageAlt` (texto alternativo), `audioTranscript` (transcripción completa si hay audio), y el texto equivalente del vídeo LSE (en el campo `videoTranscript`, que puede ser transcripción o una descripción fiel del contenido signado). Operativamente, esto obliga a la entidad a trabajar con un estándar editorial: no basta con “subir un mp3”; hay que acompañarlo de su texto equivalente para que el contenido sea usable por distintas personas.

Los enlaces a multimedia siguen un patrón deliberado: en lugar de enlazar a “una URL externa cualquiera”, se referencia siempre la ruta interna `/api/media/...`. Eso convierte el multimedia en parte del sistema, no en un conjunto de enlaces frágiles. En la práctica, el contenido hace algo como:

- `"/api/media/mi-ruta/parada-1.mp3"`
- `"/api/media/mi-ruta/parada-1.mp4"`
- `"/api/media/mi-ruta/parada-1.jpg"`

Y lo único que debe asegurar quien replica es que esos archivos existan en App Storage con el mismo nombre/ruta. Si se cambia el nombre en storage, hay que cambiar el JSON; si no, el usuario final verá un “archivo no encontrado”.

Un detalle que facilita mucho la transferencia es que los ficheros de ruta incluyen un bloque `_help`. Ese bloque actúa como “documentación incrustada” para que cualquiera que edite el contenido recuerde qué significa cada campo. Y lo importante: el servidor elimina `_help` automáticamente al entregar el JSON, así que no contamina los datos reales que consume la web. Esto es muy útil cuando el contenido lo toca alguien que no es desarrollador: el fichero se convierte en una plantilla autoexplicativa.

En conjunto, este diseño consigue algo muy difícil en proyectos municipales: que publicar una ruta nueva sea, esencialmente, un trabajo de edición y carga de archivos, no un trabajo de programación. El repositorio se convierte en la “fuente de verdad” versionable, y cada cambio queda registrado (quién lo hizo, cuándo y qué se cambió), lo cual es exactamente lo que necesitas cuando la plataforma tiene que vivir más allá del equipo inicial.

7) Cómo se gestiona el multimedia en App Storage

En PAUDIT, los audios, vídeos e imágenes no se enlazan como “archivos sueltos” repartidos por ahí, sino como parte del propio sistema. Esto es intencional y muy importante para la transferencia: cuando alguien abre una parada en el móvil y pulsa “play”, lo último que queremos es que el contenido falle porque el archivo está en un sitio distinto, porque se ha cambiado el nombre o porque el navegador no puede pedir correctamente fragmentos del vídeo.

Por eso, en los ficheros de contenido (los JSON de cada ruta) el multimedia no se referencia con enlaces externos, sino con rutas internas del tipo:

/api/media/mi-ruta/parada-1.mp3

Estas rutas internas funcionan como un “contrato” muy claro: la web pide ese archivo al servidor de PAUDIT, y el servidor lo busca en App Storage con esa misma ruta (la “clave” del objeto). Si el objeto existe, se sirve. Si no existe, devuelve un 404. Dicho de forma simple: si el nombre coincide, funciona; si no coincide, falla. Y eso es una ventaja, porque convierte los fallos en algo fácil de diagnosticar.

Para que esta mecánica sea estable en el tiempo y transferible a otros equipos, lo más recomendable es adoptar una convención de nombres sencilla, coherente y sin sorpresas. Un patrón que funciona bien es organizar por slug de ruta y nombrar por parada:

- mi-ruta/portada.jpg
- mi-ruta/parada-01.jpg
- mi-ruta/parada-01.mp3
- mi-ruta/parada-01.mp4

La regla práctica aquí es no improvisar. Si hoy se sube “parada1.mp3” y mañana alguien sube “parada-1-final.mp3”, el sistema seguirá funcionando mientras el JSON apunte bien, sí, pero a medio plazo se vuelve difícil de mantener. En cambio, cuando la convención está fijada, cualquier persona nueva en el equipo entiende dónde va cada archivo, y es mucho más difícil cometer errores.

Otro punto operativo importante es entender por qué PAUDIT sirve el multimedia “a través del servidor” y no como enlaces directos a storage. En audio y vídeo, los navegadores modernos suelen hacer peticiones parciales (por ejemplo, para avanzar a un minuto concreto). Si el servidor no entrega bien esos rangos, el usuario nota cortes, esperas o fallos de reproducción. El endpoint /api/media/... está precisamente para que la reproducción sea sólida en móvil: el servidor hace de intermediario y entrega el archivo de forma compatible con lo que espera el navegador.

En el día a día, este enfoque se traduce en una rutina muy simple cuando se publica una ruta nueva: primero se suben los archivos a App Storage con los nombres definitivos, y después se rellenan los JSON usando exactamente esas rutas. Si en algún momento hay que reemplazar un audio o un vídeo, lo ideal es mantener el mismo nombre (misma clave) y sustituir el archivo; así no hay que tocar el contenido ni se generan inconsistencias.

8) Cómo se publica en producción

Una vez que PAUDIT funciona en desarrollo, el siguiente paso es convertirlo en un servicio público estable. El criterio de transferencia aquí no es “que se pueda desplegar una vez”, sino que se pueda operar: publicar cambios de contenido, arreglar errores si aparecen y mantener el servicio sin que cada actualización sea un proyecto en sí misma.

La publicación en Replit se apoya en un flujo que, en la práctica, conviene interiorizar como una secuencia repetible: se construye la aplicación, se ejecuta en modo producción y se expone como un deployment.

El proyecto ya trae preparada esa separación entre desarrollo y producción para que lo que se ejecute “de cara al público” sea un build reproducible y no un entorno de trabajo. Esto ayuda mucho a la estabilidad: lo que está publicado es exactamente lo que se construyó desde el repositorio en ese momento.

Con el deployment en marcha, el siguiente salto habitual en una réplica municipal es el dominio propio: que la plataforma esté bajo un dominio oficial (por ejemplo, rutas.turismo-municipio.es o similar). Operativamente esto no es una cuestión de programación, sino de coordinación administrativa: hay que configurar en Replit el dominio y después aplicar los registros DNS que indique (normalmente registros A y verificación). El punto importante para la transferencia es que el dominio quede documentado internamente (quién lo gestiona, dónde se cambia el DNS, credenciales, contacto del registrador), porque si se pierde esa trazabilidad se complica la continuidad del servicio.

A nivel de operación cotidiana, PAUDIT se mantiene con una lógica muy “editorial”: lo que más se publica son cambios en contenido, no cambios en software. Por eso, el día que una entidad publique una ruta nueva o actualice una existente, el control de calidad debería centrarse en lo que realmente rompe la experiencia del visitante:

- que el listado muestre la ruta y su portada,
- que el detalle cargue sin errores,
- que todas las paradas se recorran bien en móvil,
- que el audio se reproduzca y la transcripción esté visible,
- que el vídeo (si existe) se reproduzca y tenga su texto equivalente,
- que las imágenes aparezcan y el texto alternativo sea correcto.
-

Hay un detalle muy práctico que suele ahorrar tiempo: cuando algo falla, probar la URL directa del multimedia. Si el JSON dice `/api/media/mi-ruta/parada-01.mp3`, abrir esa URL en el navegador te dice inmediatamente si el problema es “no existe el archivo” (404) o “existe, pero el reproductor no lo carga por otra razón”. Este tipo de diagnóstico es especialmente útil cuando el equipo es mixto y no todo el mundo tiene que entrar a revisar logs.

Finalmente, para que la operación sea sostenible de verdad, lo recomendable es que la entidad establezca un hábito: cada publicación de contenido debe dejar una huella clara en el repositorio (commit con mensaje descriptivo) y, si se trabaja con varias personas, que los cambios pasen por revisión. No hace falta burocracia; basta con que alguien revise que los enlaces a multimedia no estén rotos y que las transcripciones estén completas. Es ese pequeño control lo que evita que una ruta publicada “a medias” se quede pública con fallos y degrade la confianza del visitante.

9) Cómo se produce contenido accesible

La accesibilidad en PAUDIT no es un sello abstracto: se juega en el contenido concreto de cada parada. La plataforma puede estar perfectamente construida, pero si una parada tiene un audio sin transcripción, un vídeo en LSE sin texto equivalente o una imagen sin descripción, la experiencia deja fuera a parte del público. Por eso, cuando se transfiere PAUDIT, lo que se transfiere también es una manera de producir contenidos con criterios claros.

La unidad de trabajo real es la parada. Y una parada “bien cerrada” suele construirse de dentro hacia fuera: primero se define qué se quiere contar (el relato), después se prepara el texto base, luego se produce el audio y el vídeo (si aplica), y por último se crean sus equivalentes textuales y los metadatos accesibles. Este orden no es casual. Si empiezas por grabar sin tener el guion pulido, acabas regrabando o llenando la transcripción de correcciones. En cambio, cuando el texto base está bien, el resto encaja con menos fricción.

En el caso del audio, el estándar mínimo es que el visitante pueda escuchar y, si lo necesita, leer exactamente lo mismo. No vale una “idea general” ni un resumen: la transcripción debe ser completa y fiel. Esto, además de accesibilidad, tiene un efecto secundario muy positivo: permite que el contenido sea indexable, reutilizable y revisable. Si mañana se detecta un dato incorrecto, se corrige el texto y se sabe qué parte del audio hay que regrabar.

En el caso del vídeo en LSE, el criterio no debería ser “poner un vídeo”, sino garantizar que el contenido del vídeo no se convierte en un canal exclusivo. Por eso PAUDIT acompaña el vídeo con un texto equivalente: una transcripción o una descripción suficientemente fiel que permita acceder al mismo contenido por lectura. En una réplica realista, este texto equivalente también es lo que permite revisar el contenido con facilidad, traducirlo si un día se necesita o adaptarlo a otros formatos (por ejemplo, Lectura Fácil).

Las imágenes también tienen un papel más importante de lo que parece. En turismo patrimonial, casi ninguna imagen es “decorativa”: normalmente aporta información. Por eso el texto alternativo no debería ser una etiqueta genérica (“foto de la iglesia”), sino una descripción breve de lo relevante (“fachada barroca con portada de...”, “torre defensiva con almenas...”, “interior con bóveda de...”). Es un detalle pequeño que, cuando se hace bien, convierte el contenido en verdaderamente inclusivo.

En cuanto al texto principal de la parada, PAUDIT funciona especialmente bien cuando se escribe pensando en móvil: bloques cortos, frases claras, una idea por párrafo. No es solo una cuestión estética; es una forma de asegurar que la lectura sea viable en movimiento y que el usuario no abandone por fatiga. Si la entidad incorpora Lectura Fácil, conviene tratarla como una versión editorial con su propio rigor: no es “acortar”, sino reescribir con estructura más simple, vocabulario directo, orden lógico y, si es posible, validación por quien corresponda.

Lo más importante para la transferencia es asumir que producir contenido accesible requiere un pequeño “cierre” antes de publicar. Ese cierre consiste en revisar tres coherencias básicas: que el texto, el audio y el vídeo (si existe) cuentan lo mismo; que los equivalentes textuales están completos; y que todos los enlaces a multimedia responden bien. Cuando esa revisión se convierte en hábito, PAUDIT deja de ser un proyecto puntual y se convierte en un sistema que puede crecer ruta a ruta sin perder calidad.

10) Cómo organizar el trabajo para que PAUDIT sea sostenible

Cuando una entidad replica PAUDIT, el reto no suele ser “levantar la web” una vez, sino conseguir que el sistema pueda seguir vivo: publicar nuevas rutas, corregir información, actualizar audios o vídeos, incorporar mejoras y, sobre todo, hacerlo sin depender de una única persona imprescindible. Por eso, la organización del trabajo es una parte central de la transferencia.

Lo más efectivo es separar responsabilidades de una forma muy natural: por un lado está el producto editorial (lo que se cuenta y cómo se cuenta), y por otro lado está la operación técnica (cómo se publica y se mantiene el servicio). En equipos municipales o comarcales, esto encaja bien porque no obliga a que “todo el mundo programe”, pero sí exige que cada publicación tenga revisión.

En la práctica, PAUDIT se sostiene con tres perfiles mínimos, que pueden ser tres personas distintas o dos si el equipo es pequeño:

- Responsable editorial. Es quien garantiza que las rutas tienen coherencia, que el relato es correcto, que el tono es apropiado, que las fuentes están bien elegidas y que lo publicado representa al enclave con rigor. Este rol decide qué entra y qué no entra, y es quien aprueba la “versión final” del texto antes de grabar audio o producir vídeo. Si el texto base está bien cerrado, el resto del proceso fluye; si no lo está, se multiplica el retrabajo.
- Responsable de accesibilidad y calidad de contenido. Es quien revisa que la accesibilidad sea real: que las transcripciones estén completas, que el vídeo LSE tenga su texto equivalente, que las imágenes tengan un alt útil, que el contenido sea legible en móvil y que no existan barreras obvias (párrafos interminables, enlaces rotos, audios sin texto, etc.). Este rol no tiene por qué ser técnico; de hecho, funciona muy bien cuando se concibe como control de calidad editorial con foco en accesibilidad.
- Responsable técnico / operador. Es quien gestiona el repositorio, los despliegues en Replit, la conexión de dominio, la configuración de App Storage y la resolución de incidencias. Su trabajo, en el día a día, no debería ser “tocar código” constantemente, sino asegurar que la publicación y operación son estables. Este rol es también quien ayuda a establecer disciplina de cambios: qué se mergea, cuándo se despliega, cómo se documentan publicaciones

Con esos tres roles claros, la colaboración se vuelve sencilla porque se puede convertir en un ritual repetible. El flujo de trabajo más sostenible para PAUDIT es el de “publicación editorial con control de versiones”, que se resume en una lógica muy simple: todo cambio relevante deja rastro, se revisa y se publica con orden. No hace falta burocracia, pero sí una mínima disciplina: si alguien cambia un JSON o sustituye un audio, ese cambio no debería quedar “en un ordenador” o “en una carpeta suelta”, sino registrado en el repositorio y probado antes de salir al público.

En términos concretos, GitHub se convierte en el cuaderno de bitácora del proyecto. Cada ruta publicada o actualizada debería corresponder a commits claros con mensajes comprensibles (“Añadida ruta X”, “Actualizado audio parada 3”, “Corrección de transcripción parada 5”, etc.). Si hay varias personas tocando contenido, lo ideal es trabajar con ramas y revisión básica antes de integrar cambios. Esto no solo mejora la calidad: también evita situaciones típicas de continuidad institucional, como perder la versión correcta del material o no saber por qué se cambió algo.

1) Incidencias típicas en replicaciones y cómo diagnosticarlas rápido

En la operación real de PAUDIT hay un patrón muy claro: casi todos los problemas que aparecen en producción se explican por una de estas tres causas: contenido mal referenciado, archivos multimedia mal ubicados o despliegue que no está sirviendo la versión esperada. Saber esto convierte la resolución de incidencias en algo relativamente mecánico, incluso para perfiles no técnicos.

Un caso típico es el de “la ruta se ve, pero el audio o el vídeo no se reproduce”. En PAUDIT esto suele ocurrir cuando el JSON apunta a un archivo que no existe en App Storage o cuyo nombre/ruta no coincide. La manera más rápida de comprobarlo es abrir directamente, en el navegador, la URL del archivo a través del endpoint: si el contenido dice `/api/media/mi-ruta/parada-01.mp3`, basta con intentar abrir esa misma URL. Si devuelve 404, el diagnóstico es inmediato: el archivo no está en la ruta correcta o se ha nombrado diferente. Si abre y descarga, el archivo existe y entonces el problema se reduce a compatibilidad de formato o a un detalle del reproductor.

Otra incidencia frecuente es “la ruta no aparece en el listado” o “al entrar da error”. Aquí normalmente el fallo está en la relación entre `routes.json` y el fichero de detalle `routes/<slug>.json`. Si el slug del listado no coincide exactamente con el nombre del fichero, el servidor no lo encontrará. También puede ocurrir que el JSON de detalle tenga un error de sintaxis (una coma fuera de lugar, comillas mal cerradas). Este tipo de error suele manifestarse como una ruta que “rompe” mientras el resto funciona. Es una de las ventajas de tener contenido por ficheros: el error queda aislado y no compromete toda la plataforma.

Un tercer caso muy habitual es el de “he cambiado cosas y no se ven en producción”. Esto casi siempre se debe a que el deployment está sirviendo una versión anterior (no se ha redeployado) o a que el cambio no está realmente en la rama que se despliega. La forma de evitarlo es convertir la publicación en un acto consciente: después de mergeear cambios en la rama principal, se redeploya y se comprueba en la URL pública, no solo en el entorno de desarrollo. También hay incidencias más “silenciosas” que afectan a la calidad: por ejemplo, una transcripción incompleta, un alt genérico o un texto excesivo en móvil. Estas no “rompen” técnicamente, pero degradan la experiencia. Por eso conviene que el responsable de accesibilidad haga una revisión rápida de cierre cada vez que se publique una ruta: recorrer paradas en móvil, abrir audio, verificar que el texto equivalente está ahí y que las imágenes tienen descripción útil.

La idea general, y esto es muy transferible, es que PAUDIT está diseñado para que el diagnóstico sea trazable: si algo falla, suele ser visible en una URL concreta, en un fichero concreto o en un despliegue concreto. Y eso es lo que permite que una entidad lo opere con garantías sin entrar en depuración compleja.

12) Cómo evolucionar PAUDIT sin perder la lógica de transferencia

PAUDIT v1.0 está intencionalmente construido para ser fácil de replicar: contenido en JSON, multimedia en App Storage y despliegue integrado. Esa simplicidad es una fortaleza, pero también marca un límite natural: cuando una entidad crece mucho en volumen de rutas o necesita que varios editores trabajen a diario, aparece la necesidad de herramientas de edición más cómodas que “editar JSON”.

La evolución más sensata no es abandonar lo que ya funciona, sino mantener los principios y cambiar lo mínimo. El principio esencial que conviene conservar es que el producto público (la web) consume un contrato de datos estable: rutas, paradas, campos accesibles, enlaces a multimedia. Mientras ese contrato se mantenga, se puede cambiar la forma de gestionar el contenido detrás.

En una evolución intermedia, una entidad puede incorporar un panel de administración sencillo que genere o edite esos mismos JSON con formularios. Eso permite que perfiles no técnicos editen sin riesgo de romper sintaxis. Otra evolución posible, cuando ya hay muchos editores o necesidad de auditoría, es usar un CMS o base de datos para almacenar rutas y paradas, pero exportando o sirviendo los datos con el mismo esquema que ya entiende el frontend. De esta forma, la web pública apenas se toca y la transferencia sigue siendo coherente.

En cuanto al multimedia, la evolución suele ir por optimización y gobernanza: establecer tamaños recomendados, perfiles de exportación para vídeo, plantillas de naming más estrictas y procedimientos de sustitución (reemplazar manteniendo la misma clave para no editar contenido). Esto evita que el coste y la complejidad crezcan sin control y mantiene una experiencia móvil buena incluso con material pesado.

Finalmente, una evolución muy valiosa, cuando se trabaja con accesibilidad de forma seria, es profesionalizar el “cierre de publicación”: checklist breve firmable, registro de versión de ruta, fecha de revisión y, si aplica, validación por entidades o colectivos. No hace falta convertirlo en burocracia, pero sí en un mecanismo de calidad que asegure que PAUDIT no pierde su razón de ser conforme crece.

En resumen: PAUDIT puede escalar y mejorar, pero conviene hacerlo preservando el núcleo transferible: contratos de contenido claros, multimedia gestionado con convención estable y un flujo de publicación reproducible. Eso es lo que permite que el sistema sea replicable hoy y sostenible mañana.